

OFDM Joint Communications and Radar (JCAS)

Technical Documentation

Repository: smanthat/OFDM | June 2026

Status: Communications path validated on hardware (BER ~0.02 on AD2 loopback).

This document is the developer reference for the system as implemented. It is written so that any future work (radar path, CFO correction, FPGA port) can build directly on it without re-deriving decisions. Read alongside the code; the code is canonical where they disagree.

1. System Overview

An OFDM physical layer designed from the start to serve two functions with one waveform:

- Communications: QPSK data over 64 subcarriers, blind frame synchronization, channel estimation, and equalization. Implemented and hardware-validated.
- Radar: range-Doppler imaging using the same frame via symbol division and 2D FFT. Designed, not yet implemented. See Section 8 (Extension Points).

End goal: real-time implementation in hand-written SystemVerilog on a Nexys A7-100T (Artix-7 xc7a100tcsg324-1), with the FFT shared between the comms and radar paths.

1.1 Signal Flow (current, comms path)

```
bits.txt (2048 bits)
-> QPSK_Mapper -> Symbol_Mapper -> [prepend 2 preambles]
-> IFFT per block -> add CP (16 samples) -> flatten -> normalize -> (quantize)
-> ad2_i_waveform.csv / ad2_q_waveform.csv
-> [AD2 AWG W1/W2, SYNCHRONIZED, Run All]
-> [physical loopback / channel]
-> [AD2 scope Ch1=I, Ch2=Q capture] -> Recieved_waveform.csv
-> Schmidl-Cox sync on RAW 1-D stream -> d_peak
-> slice 18 blocks from d_peak -> strip CP -> FFT per block
-> H_est = freq[1] / channel_preamble (full-spectrum preamble)
-> data_eq = freq[2:] / H_est (zero-forcing equalize)
-> Demapper -> BER vs bits.txt
```

2. Physical Layer Parameters

Parameter	Value	Notes
Subcarriers N	64	FFT size
Bandwidth	500 kHz	Limited by AD2 AWG practical rate
Sample rate	500 kS/s	Complex baseband
Subcarrier spacing	7.8125 kHz	BANDWIDTH / N
OFDM symbol duration	128 us	N / SAMPLE_RATE
CP length CP_LEN	16 samples (32 us)	Hardcoded, Formula: $\text{int}(\text{np.ceil}(T_{\text{GI}} * \text{BANDWIDTH}))$ Must exceed channel delay spread; sim channel max tap = 7

Block length BLOCK_LEN	80 samples	N + CP_LEN
Modulation	QPSK, Gray-coded	2 bits per symbol
Frame	18 blocks	Hardcoded, 1 timing preamble + 1 channel preamble + 16 data
Samples per frame	1440	18 x 80
Payload	2048 bits	Exactly 16 x 64 x 2 - must divide evenly (Section 7.5)
Fixed-point budget	W=16, F=13	From BER-vs-bit-width sweep (Section 6)

Invariant: payload bits = 2 x N x NUM_DATA_SYMBOLS. Any other length silently desynchronizes the frame (bug 7.5).

3. Frame Structure

```

block index :    0          1          2 ... 17
contents    : [TIMING]  [CHANNEL]  [DATA x 16]
               (SC sync) (H_est)

```

Each block on the wire: [CP: last 16 samples of symbol][64-sample IFFT output].

3.1 Timing Preamble (block 0)

- Pseudo-random QPSK on even subcarriers only; odd subcarriers zero.
- Even-only spectrum produces a time-domain signal with two identical halves: $x[n] = x[n + N/2]$. This is the property Schmidl-Cox detects.
- PN (not constant) values spread energy over all 64 samples. A constant value (e.g. $1+1j$ on every even bin) produces an impulse train - roughly 97% of samples near zero - which destroys the sync metric and stresses the DAC (bug 7.3).
- Even bins scaled by $\sqrt{2}$ to restore full symbol power (half the bins are off; power scales as amplitude squared, so amplitude x $\sqrt{2}$ doubles the power).

3.2 Channel Preamble (block 1)

- Pseudo-random QPSK on all 64 subcarriers - no zero bins, so the receiver division $H_est = freq[1] / channel_preamble$ is defined on every subcarrier (bug 7.4).
- Uses a different seed from the timing preamble so the two are statistically independent (prevents the timing correlator from partially locking onto block 1).

3.3 Seeds

Defined once in preamble_gen.py and imported by both transmitter and receiver:

```

SC_SEED = 0xB00B3E5    # timing preamble
CH_SEED = 0xDEADBEEF   # channel preamble

```

If TX and RX ever generate preambles from separate code paths, they will eventually disagree. Keep generation in one shared module - this is a hard rule.

4. Module Reference

4.1 preamble_gen.py

Shared preamble generation. Imported by TX and RX.

```
timing_preamble_gen(N) -> complex ndarray (N,)
# PN-QPSK on even bins x sqrt(2), zeros on odd bins.
# Property: ifft(out)[:N//2] == ifft(out)[N//2:]
```

```
channel_preamble_gen(N) -> complex ndarray (N,)
# PN-QPSK on all bins. No zeros.
```

`_QPSK_MAP = [1+1j, 1-1j, -1+1j, -1-1j]` is internal to preambles only; it is NOT the same ordering as the data mapper and does not need to be (preambles only need TX/RX self-consistency, not Gray-code structure).

4.2 QPSK_Mapper.py / Demapper.py

Data-path bit-to-symbol conversion. Gray-coded, quadrant-based:

Bits	Symbol	Quadrant
00	+1+1j	I >= 0, Q >= 0
01	-1+1j	I < 0, Q >= 0
11	-1-1j	I < 0, Q < 0
10	+1-1j	I >= 0, Q < 0

These two files are a matched pair - round-trip verified exact (BER 0.0 on perfect symbols). Do not change one without the other. The demapper makes hard decisions by sign of real/imag, which is equivalent to nearest-constellation-point for QPSK; no separate snapping step is needed before it (Section 7.7).

`demapper(signals, original_len, original_len_symbol)` truncates output to `original_len` bits, which removes pad bits added by `symbol_mapper`.

4.3 Symbol_Mapper.py

`symbol_mapper(bits, N)` reshapes a 1-D complex symbol stream into `(num_symbols, N)` rows, zero-padding the tail to a multiple of `N`. Returns `(N_symbols, original_len_symbol)`. The pad symbols are zeros - they decode to garbage bits, which is why the demapper truncation to `original_len` matters.

4.4 transmitter_main.py

Top-level TX. Key sections in order:

- Parameters (Section 2). `NUM_BLOCKS = 18` is currently hardcoded - recommended fix is `NUM_BLOCKS = 2 + N_symbols.shape[0]` so a wrong-length `bits.txt` cannot silently desync the frame (bug 7.5).
- Bits -> QPSK -> (16, 64) data matrix.
- `vstack(timing_preamble, channel_preamble, data)` -> (18, 64).
- `np.fft.ifft(..., axis=1)` -> time domain per block.
- CP prepend: `concatenate((block[:, -CP_LEN:], block), axis=1)` -> (18, 80).
- Flatten -> scale to 0.8 x max for AD2 voltage range.
- Optional fixed-point quantization (`dec_to_fixed_point`, `W=16`, `F=13`). Note: the current file has `i_wave_fixed = i_wave` (quantization bypassed) and `F = 10`. For the float hardware path that is fine; for fixed-point experiments re-enable the call and set `F = 13`.
- Write `ad2_i_waveform.csv`, `ad2_q_waveform.csv`, and `transmitted_bits.txt` with the 7-field metadata footer.

```
original_length,original_len_symbol,N,BANDWIDTH,CP_LEN,TS,NUM_BLOCKS
2048,1024,64,500000,16,2e-06,18
```

The metadata footer is the TX-to-RX contract. The receiver parses fields by index; changing the format requires changing both files.

Self-check (prints True when the SC property holds):

```
preamble_time = np.fft.ifft(preamble_freq, axis=1)
np.allclose(preamble_time[0, :32], preamble_time[0, 32:]) # row 0, COLUMN halves
```

Note the indexing: [0, :32] is row 0, first 32 samples. `preamble_time[0:32]` (a row slice) is meaningless here and was a recurring bug (Section 7.1 family).

4.5 receiver_communication.py

Top-level RX. Stages:

(a) Load capture

Reads Recieved_waveform.csv (WaveForms scope export, #-prefixed header comments). Ch1 -> I, Ch2 -> Q. NaN rows masked. Produces the raw complex stream `r`.

(b) Schmidl-Cox sync - on the raw 1-D stream, before any reshape

```
L = N // 2
num = len(r) - N # == len(r) - 2L since N = 2L
for d in range(num):
    p = sum_m conj(r[d+m]) * r[d+m+L] # m = 0..L-1
    r_c = sum_m |r[d+m+L]|^2
    metric[d] = |p|^2 / (r_c^2 + 1e-12)
```

- `p`: when the window sits on the preamble, every term is positive-real and the sum is large; elsewhere terms have random phase and cancel.
- metric is in approximately [0, 1], amplitude-invariant, and CFO-immune for timing.
- The metric plateaus over approximately CP_LEN samples (the CP region also satisfies the half-repeat property); any `d_peak` on the plateau lands inside the CP, which is why a few samples of timing error are harmless.

Energy gating (rejects false peaks where `r_c` near 0 makes the ratio blow up):

```
typical = np.median(energy[energy > 0])
metric[energy < 0.2 * typical] = 0 # ZERO OUT - do not boolean-filter!
d_peak = int(np.argmax(metric))
```

Zeroing keeps the array full-length so `d_peak` remains a valid sample index. Boolean filtering (`metric = metric[mask]`) shrinks the array and breaks the index-to-sample correspondence (bug 7.6).

(c) Frame slice and FFT

```
frame = r[d_peak : d_peak + BLOCK_LEN * NUM_BLOCKS] # NUM_BLOCKS from metadata
assert len(frame) == BLOCK_LEN * NUM_BLOCKS # short-frame guard
freq = np.fft.fft(frame.reshape(-1, BLOCK_LEN)[: , CP_LEN:], axis=1) # (18, 64)
```

(d) Channel estimate and equalize

```
H_est = freq[1] / channel_preamble_gen(N) # row 1 = channel preamble
data_eq = freq[2:] / (H_est[None, :] + 1e-12) # rows 2..17 = data
```

Row map: `freq[0]` timing preamble (sync only - never divide by it), `freq[1]` channel preamble, `freq[2:]` data. An off-by-one here gives BER 0.5 with no crash.

(e) Demap and BER

`data_eq.flatten()[:original_len_symbol]` -> demapper -> compare to `bits.txt`.

Healthy run signature (hardware): `metric_peak` ≥ 0.7 , median ~ 0.02 (at least 25x ratio), non-finite `H_est` bins = 0, BER ~ 0.02 .

4.6 Multipath_channel.py

Simulation channel for offline validation: 3 taps $h[0]=1.0$, $h[3]=0.5$, $h[7]=0.2$ plus AWGN at configurable SNR. Max delay 7 samples means CP_LEN = 16 is sufficient. Returns (rx_signal, H, h).

4.7 fixed_point_reference.py

dec_to_fixed_point(x, F, W): scale by 2^F , np.round, saturate to $[-2^{(W-1)}, 2^{(W-1)}-1]$, return ints. Requirements baked in from earlier debugging: binary (power-of-two) scaling, np.round not int() truncation, explicit saturation. (fixed_to_decimal.py is an empty stub - the inverse, $x / 2^F$, still needs writing for the fixed-point receive path.)

5. Hardware Procedure (AD2 + WaveForms)

The single most failure-prone part of the workflow. Follow exactly.

5.1 Transmit Setup

- Run transmitter_main.py. Verify console prints frame shape (18, 80) and the metadata footer ends in ",18".
- WaveForms Wavegen: W1 (Channel 1): Custom -> import ad2_i_waveform.csv. W2 (Channel 2): Custom -> import ad2_q_waveform.csv.
- Both channels: Sample Rate 500 kHz, Run continuous / Repeat infinite, Amplitude 1 V.
- Channels dropdown: Synchronized. Without this the two AWGs free-run and I/Q are offset by an arbitrary number of samples (bug 7.8).
- Start with Run All (top-left). NEVER the per-channel Run buttons - individually started channels begin at different times even in Synchronized mode.

5.2 Capture

- Scope Ch1 from W1 (I), Ch2 from W2 (Q). Scope channels are inherently simultaneous; the skew risk is purely on the generation side.
- 500 kS/s, at least 4096 samples (2-3 frame repetitions; 8192 typical).
- Export -> Recieved_waveform.csv.

5.3 The I/Q Alignment Check

Run this whenever BER is mysteriously 0.5:

```
ci = np.correlate(i_capture, i_waveform, 'valid')
cq = np.correlate(q_capture, q_waveform, 'valid')
bi, bq = np.argmax(np.abs(ci)), np.argmax(np.abs(cq))
# REQUIRED: bi == bq (skew 0), both correlation signs positive,
# both peaks strong, swap-test (i vs q_waveform) peaks ~10x weaker.
```

Observation	Diagnosis
Skew much greater than 0	AWG channels not started together -> Section 5.1 last step
Swap-test peaks strong	Scope/AWG cables crossed (Ch1 and Ch2 swapped)
Negative correlation sign	A channel inverted (cabling or amplitude sign)
Both peaks weak	Wrong waveform loaded on AD2, or stale capture

6. Fixed-Point Budget

From the BER-vs-(W, F) sweep against the float64 golden chain:

- W = 16, F = 13 is the adopted format (Q2.13 plus sign).
- F = 14 leaves 1 integer bit -> saturation risk on PAPR peaks. F <= 12 raises the quantization noise floor visibly in BER.
- Quantize only at RTL module boundaries (TX output, RX input), never inside blocks that model vendor IP (the FFT) - those have their own internal scaling schedules decided at HDL time.

Open HDL-time decisions that interact with this budget: FFT per-stage scaling vs block floating point; storing $1/H_{\text{est}}$ once per frame to avoid dividers in the equalizer datapath; bit growth in the SC correlator accumulators (P and R need their own width analysis - $|P|^2$ doubles the bit width).

7. Bug Ledger (what failed, why, and the rule that prevents it)

Each of these cost real debugging time. The rule is the thing to carry forward.

#	Bug	Symptom	Root cause	Rule
7.1	Sync run on reshaped/FFT data	BER 0.5, garbage d_peak	Reshape assumes sample 0 is a frame boundary - circular	Sync lives in sample-space. Metric runs on the flat 1-D stream; reshape only after d_peak exists
7.2	CP_LEN = ceil(0.8e-6 x 500e3) = 1	ISI even with perfect sync	Guard time shorter than one sample period	CP_LEN must exceed channel delay spread in samples; validate against the channel model
7.3	Constant 1+1j preamble	Flat sync metric (peak 1.3 vs median 0.84)	Impulse-train time signal; ~97% zero samples	Preambles use PN sequences, never constants
7.4	H_est divided by timing preamble	inf/nan on all odd bins	Timing preamble has zero odd bins	Channel estimation needs a full-spectrum preamble (or interpolation)
7.5	bits.txt = 2065 bits	BER 0.5, no crash	17 data symbols generated, metadata claimed 18	Payload = 2 x N x NUM_DATA exactly; compute NUM_BLOCKS from N_symbols.shape[0]
7.6	metric = metric[energy > thr]	Wrong d_peak, silent	Boolean filter shrinks array, breaks index-to-sample map	Gate by zeroing in place, never by filtering
7.7	Manual constellation snapping loop	Crash / row-vs-symbol confusion	data_eq[x] indexes rows of (16, 64), not symbols	Demapper already does nearest-quadrant; no pre-snapping. Flatten before symbol-level ops
7.8	I/Q skew 1039 samples	BER 0.5, blob constellation, I-I and Q-Q each correlate fine	AWG channels started independently	Synchronized mode AND Run All; verify with Section 5.3
7.9	Stale capture / stale code	Any of the above, intermittently	TX changed without regenerate-reload-recapture	Any TX change: regenerate CSVs, reload AD2, recapture, then run RX. Metadata field count is the version marker

Meta-pattern across 7.1 / 7.6 / 7.7: layer confusion - applying sample-level operations to block-level arrays or vice versa. When indexing anything, state to yourself whether the index is a sample, a block/row, or a subcarrier/column.

8. Extension Points (how to build on this)

8.1 Radar Path (next, in progress)

The frame already carries everything radar needs. Implementation plan:

```
# Y[k, n]: received freq-domain data symbols -> freq[2:].T (64 subcarriers x 16 symbols)
# X[k, n]: KNOWN transmitted data symbols -> rebuild from bits.txt via mapper
H_radar = Y / X # symbol division: strips data modulation
rd_map = np.fft.ifft( # 2D transform:
    np.fft.fft(H_radar, axis=1), # across symbols -> Doppler axis
    axis=0) # across subcarriers -> range axis
# |rd_map|^2 peaks = targets at (range bin, Doppler bin)
```

Key numbers at current parameters:

- Range resolution $c / (2 \times BW) = 300$ m per bin (64 bins -> 19.2 km unambiguous). Fine for algorithm validation with synthetic delays; useless for real indoor targets - that requires more bandwidth (SDR front-end, Section 8.4).
- Doppler resolution $1 / (\text{NUM_DATA} \times T_{\text{block}}) = 1 / (16 \times 160 \text{ us}) \sim 488$ Hz per bin.
- Validation target: synthetic 2-target channel (distinct delay + Doppler), confirm two separated peaks in the map. This is the radar equivalent of "BER -> 0".

Keep `compute_range_doppler(Y, X) -> rd_map` as a pure function - it becomes the ML feature extractor later (Section 8.5) and the HDL block spec.

8.2 CFO Correction (closes the 2% BER gap)

The sync correlator already computes the CFO estimate for free:

```
eps = np.angle(P[d_peak]) / L # phase per sample
r_corrected = r * np.exp(-1j * eps * np.arange(len(r)))
```

Apply before the frame slice. Diagnose first: if per-symbol BER grows across the frame, it is residual phase drift (CFO); if flat, it is noise-limited and CFO correction will not help much.

8.3 Recursive Schmidl-Cox (required before HDL)

The $O(L)$ -per-sample loop is simulation-only. The streaming form is $O(1)$ per sample:

```
P(d+1) = P(d) + conj(r[d+L]) * r[d+2L] - conj(r[d]) * r[d+L]
R(d+1) = R(d) + |r[d+2L]|^2 - |r[d+L]|^2
```

Two delay lines (length L and $2L$) plus two accumulators. On FPGA, replace the division in $M(d)$ with a threshold compare: detect $|P|^2 > k \times R^2$ (one multiply, one compare). Pick k empirically in Python now (somewhere below the observed plateau height, well above the noise floor - current hardware numbers suggest $k = 0.3$ to 0.5).

8.4 Wideband RF Front-End

The AD2 caps the project at 500 kHz baseband. Next hardware step: SDR (HackRF ~ 20 MHz BW at 2.4/5.8 GHz, or PlutoSDR). 20 MHz gives 7.5 m range bins - real targets become visible. The Python chain is carrier-agnostic; only sample rate, N , and CP_LEN re-derivation are needed (CP must still exceed the new channel delay spread in the new sample units).

8.5 ML Hooks (far future, but shapes interfaces now)

- Classification operates on `|rd_map|` images (or range-Doppler-time stacks) - hence the pure-function requirement in Section 8.1.

- Data generation = the existing simulation chain in a loop with randomized target parameters plus labels. Build it as a generator, not a script.
- Keep detection (CFAR or learned) as a separate stage after `rd_map` so it can be swapped.

8.6 HDL Block Order (when the FPGA phase starts)

Smallest-first, each diffed against the Python golden model at $W=16/F=13$: mapper/demapper -> recursive SC correlator -> FFT (shared-resource design: comms needs 1 FFT per symbol, radar needs FFTs across the frame; at 500 kS/s one pipelined FFT serves both with large margin) -> channel estimator (compute $1/H$ once, multiply thereafter) -> equalizer -> symbol-division plus 2D transform.

9. Reproducibility Checklist

- `bits.txt`: exactly 2048 chars of 0/1, single line, no trailing whitespace.
- `python transmitter_main.py` -> frame (18, 80), metadata ends ",18".
- WaveForms: load both CSVs, Synchronized, 500 kHz, Repeat infinite, Run All.
- Scope: both channels, 500 kS/s, at least 4096 samples -> `Recieved_waveform.csv`.
- `python receiver_communication.py` -> expect `metric_peak` ≥ 0.7 , median ~ 0.02 , non-finite H bins: 0, BER ≤ 0.05 .
- If BER ~ 0.5 -> run the Section 5.3 I/Q alignment check before touching any code.

10. Glossary

Term	Meaning here
<code>d / d_peak</code>	Candidate / detected frame-start sample index in the raw stream
<code>L</code>	Half-symbol length, $N/2 = 32$
<code>P(d), R(d), M(d)</code>	SC half-correlation, second-half energy, normalized timing metric
plateau	Flat top of <code>M(d)</code> , width $\sim \text{CP_LEN}$; any point on it is a valid frame start
symbol division	Y/X - removing known data modulation to expose the channel (radar step)
sample-space	The flat 1-D received stream, before any block structure exists
golden model	The float64 Python chain; every fixed-point/HDL block is diffed against it

End of documentation. The Python implementation in the `smantthat/OFDM` repository is canonical; this document supplements it.